Google Cloud H2S

AI Builder Cup

Build in JAPAC.
Build for the world.



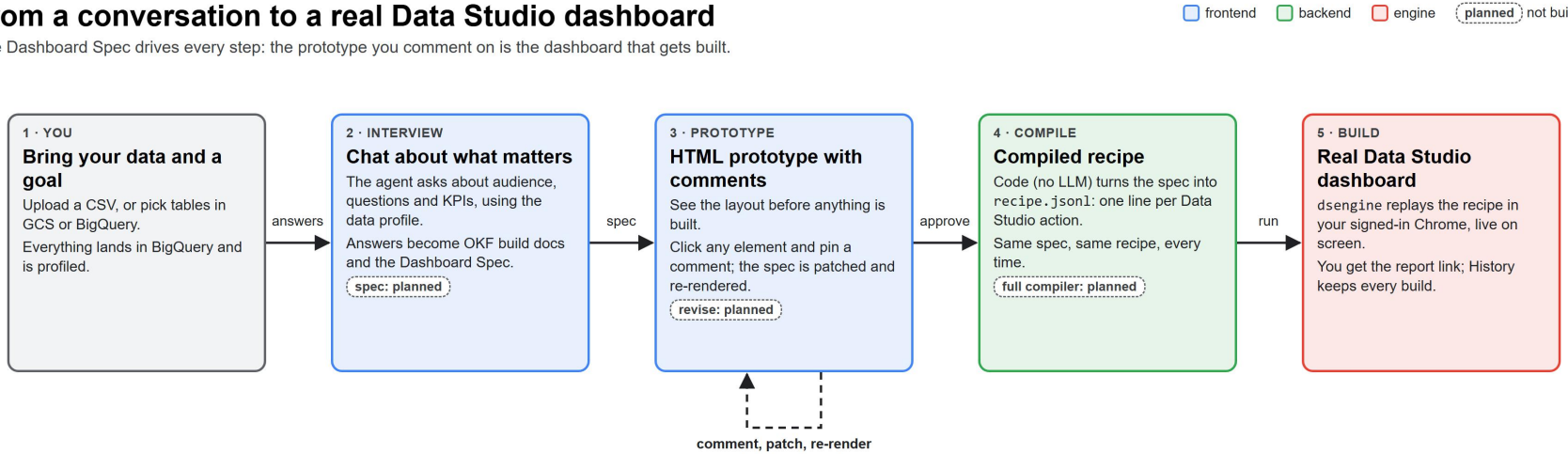
Team Details

1. Team name: Prisma
2. Team leader name: Jan Daniel Gonzalvo
3. Problem Statement: Data Studio dashboards take hours of clicks and design skill, but we will democratize it with this solution.

Brief about the idea

From a conversation to a real Data Studio dashboard

One Dashboard Spec drives every step: the prototype you comment on is the dashboard that gets built.



Why it works

The model writes a small JSON spec, never pixels or clicks. Layout, prototype and recipe are computed from that spec by code, so edits stay cheap and the build is repeatable. Planner model on plan and spec; a fast worker model everywhere else.

Opportunities

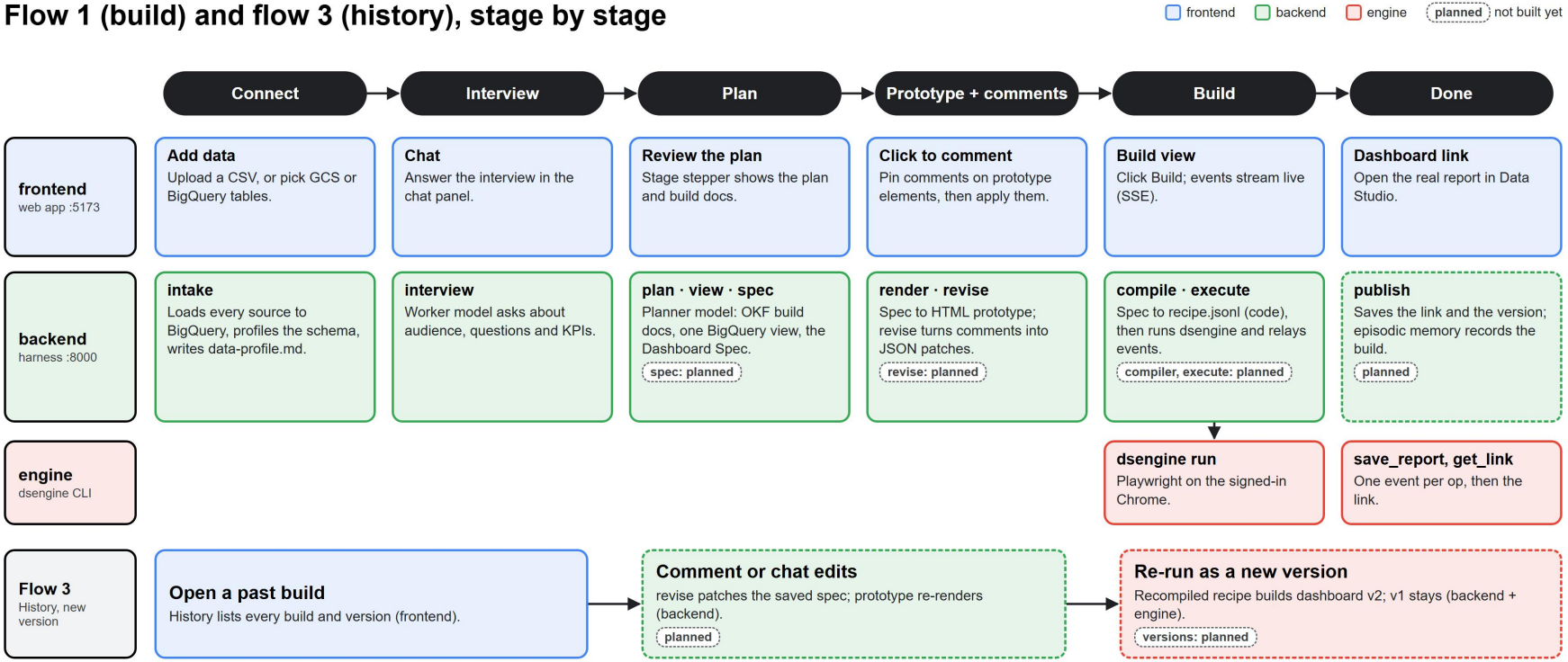
- Different: other tools generate charts or SQL; we build a real, editable Data Studio report from a compiled recipe.
- How it solves the problem: a short interview turns business questions into a Dashboard Spec; click-to-comment fixes the design before any build.
- USP: an agent harness with three memories built to learn from every build (retrieval, consolidation: planned), plus a deterministic engine that clicks Data Studio for you.
- Opportunity: every team on Google Workspace already has Data Studio and BigQuery.

List of features offered by the solution

- Data in: CSV upload, CSV in GCS or BigQuery tables, all landing in BigQuery with a profiled schema.
- Chat interview about audience, questions and KPIs; four dashboard archetypes as skills.
- HTML prototype with click-to-comment edits.
- Closed catalog of 12 Data Studio component types.
- One-click build: compiled recipe, live build events, dashboard link (full compiler: planned).
- History of builds; versions and Cloud Run hosting behind an access code: planned.

Process flow diagram or Use-case diagram

Flow 1 (build) and flow 3 (history), stage by stage



Wireframes/Mock diagrams of the proposed solution (optional)

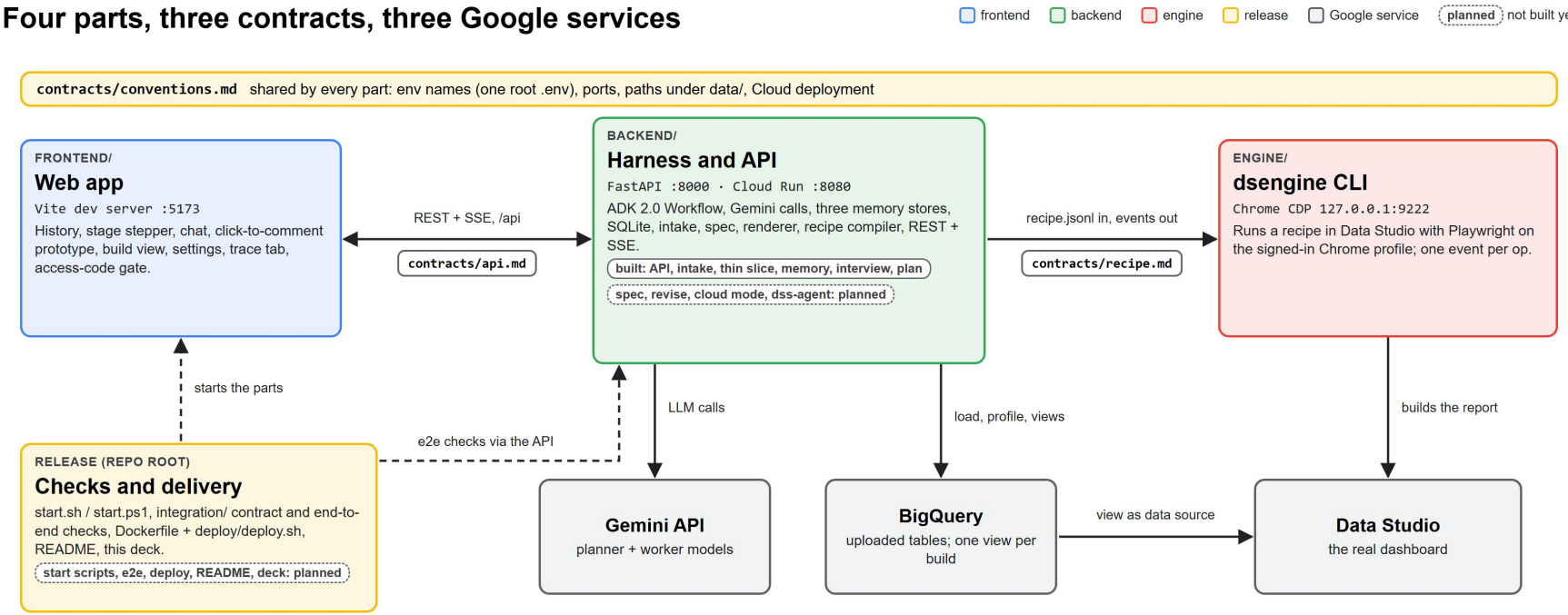
- Left: History of past builds. Center: the workspace with a stage stepper (Connect, Interview, Plan, Prototype, Build, Done).
- Chat panel beside an HTML prototype of the dashboard; click any element to pin a comment, then apply all comments at once.
- Build view streams each engine step and ends with the Data Studio link.

Architecture diagram of the proposed solution

- Four parts with written contracts: the web app (frontend), the harness and API (backend), the dsengine CLI (engine) and the release checks.
- The harness runs an ADK 2.0 graph with Gemini planner and worker models; three memories (episodic retrieval: planned).
- Data lives in BigQuery: one SQL view per build.
- The engine replays a compiled recipe in Data Studio with Playwright.
- The next six slides open each layer.

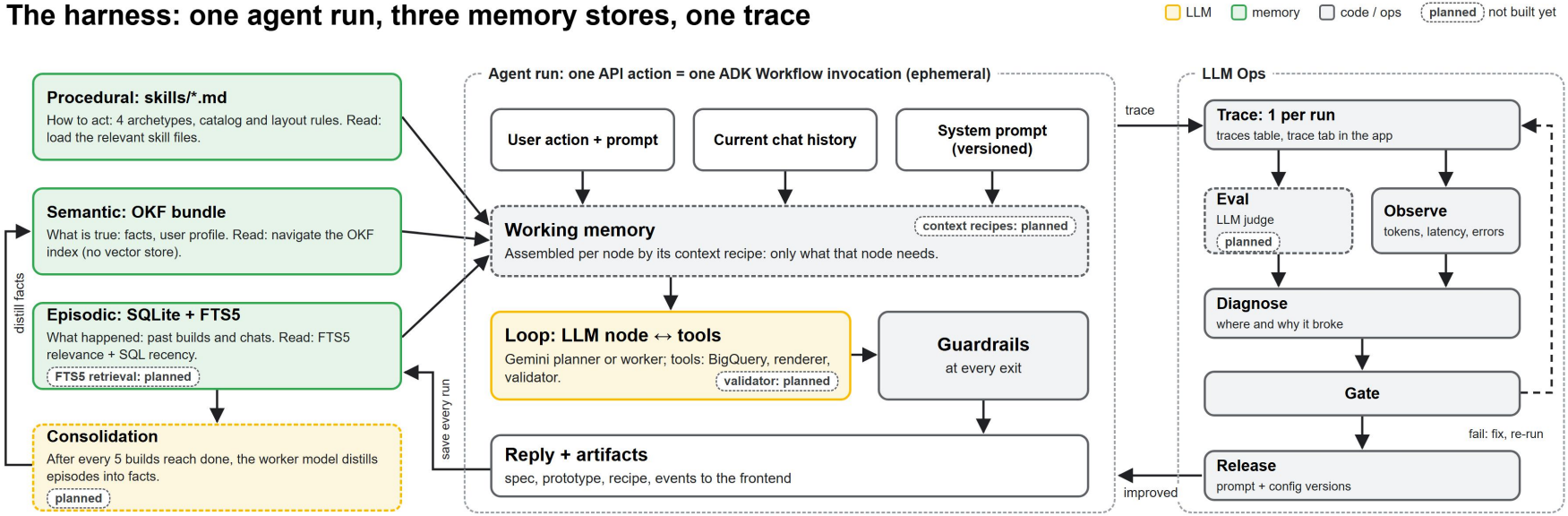
Parts and data flow

Four parts, three contracts, three Google services



The harness: three memories

The harness: one agent run, three memory stores, one trace

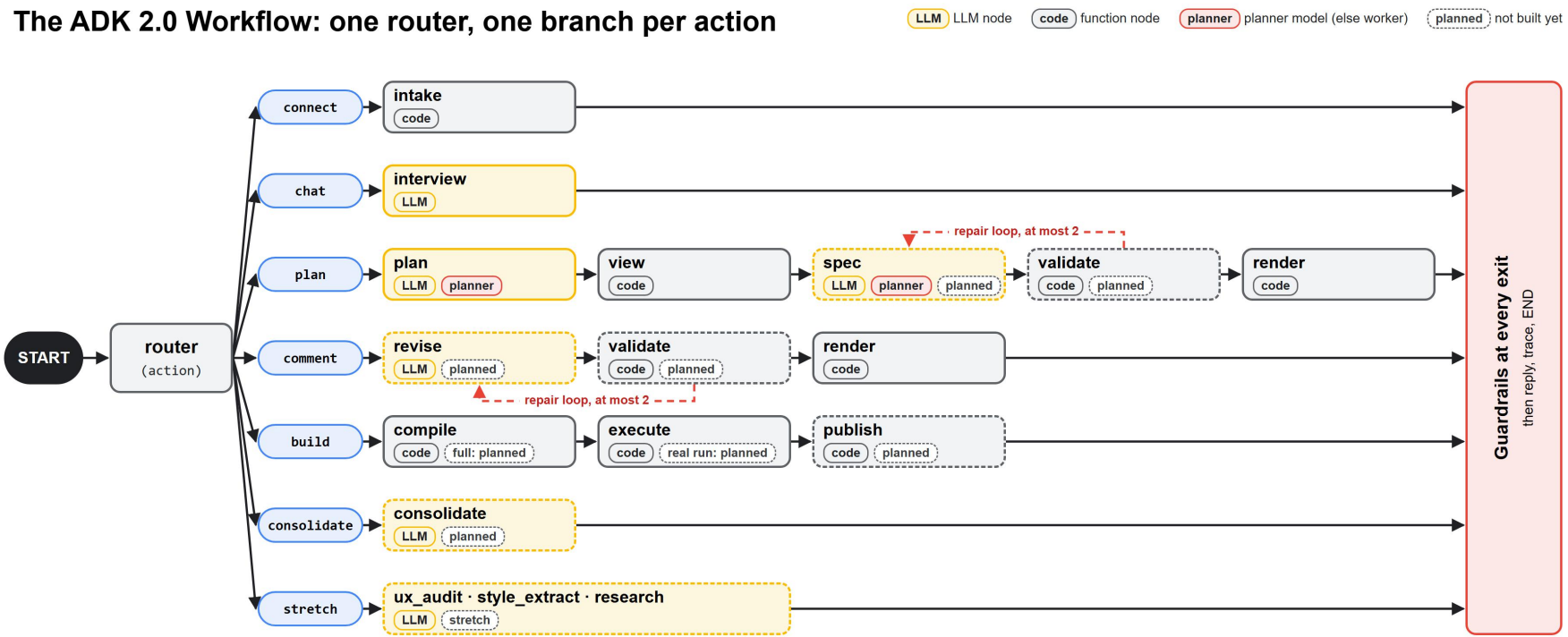


Mapped to this build

Harness = ADK 2.0 Workflow. Planner `gemini-4-argon` on plan and spec; worker `gemini-3.8-flash` on every other LLM node and on consolidation. Episodic is written every run; semantic only by consolidation; the loop exits only through guardrails.

ADK graph and repair loop

The ADK 2.0 Workflow: one router, one branch per action



Token economy

Eight token levers: the model writes decisions, code does the rest

planned not built yet

1

Spec as the single truth

The Dashboard Spec (JSON) drives the prototype, the recipe and every edit. The model never rewrites HTML or Data Studio steps.

Saves: one small JSON per turn instead of regenerated pages

spec node: planned

2

Grid layout

The spec places items on a 12-column grid; code computes every pixel box for the prototype and the recipe.

Saves: the model never outputs coordinates

3

JSON-patch comments

Each pinned comment becomes a small JSON patch on the spec, not a rewrite of the whole design.

Saves: an edit costs about the size of the change

revise: planned

4

Compiled recipe

A deterministic compiler turns the spec into recipe.jsonl; dsengine replays it with no LLM calls.

Saves: a build, and every re-run, costs zero tokens

full compiler: planned

5

Background image

Static styling (cards, headers, colors) is baked into one background PNG per page.

Saves: dozens of styling clicks become one op

background: planned

6

Per-node context recipes

Each node gets only the memory it needs: data profile, a spec slice, the relevant skill.

Saves: short prompts, no full history in every call

planned

7

Model routing

The planner model runs only on plan and spec; the fast worker model runs every other LLM node.

Saves: the expensive model on two nodes only

8

OKF instead of RAG

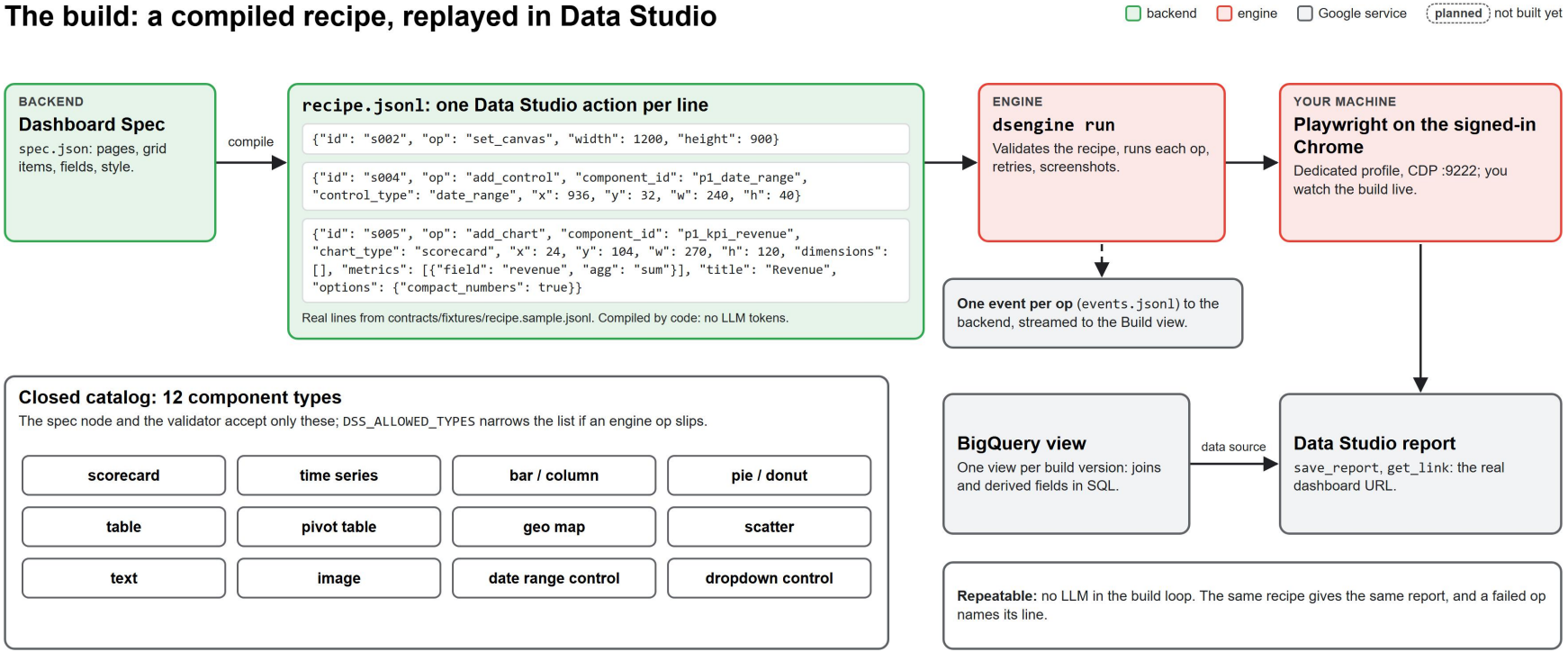
Semantic memory is an OKF bundle the model navigates by its index; no embeddings, no vector store.

Saves: no embedding calls, no retrieval noise

node context: planned

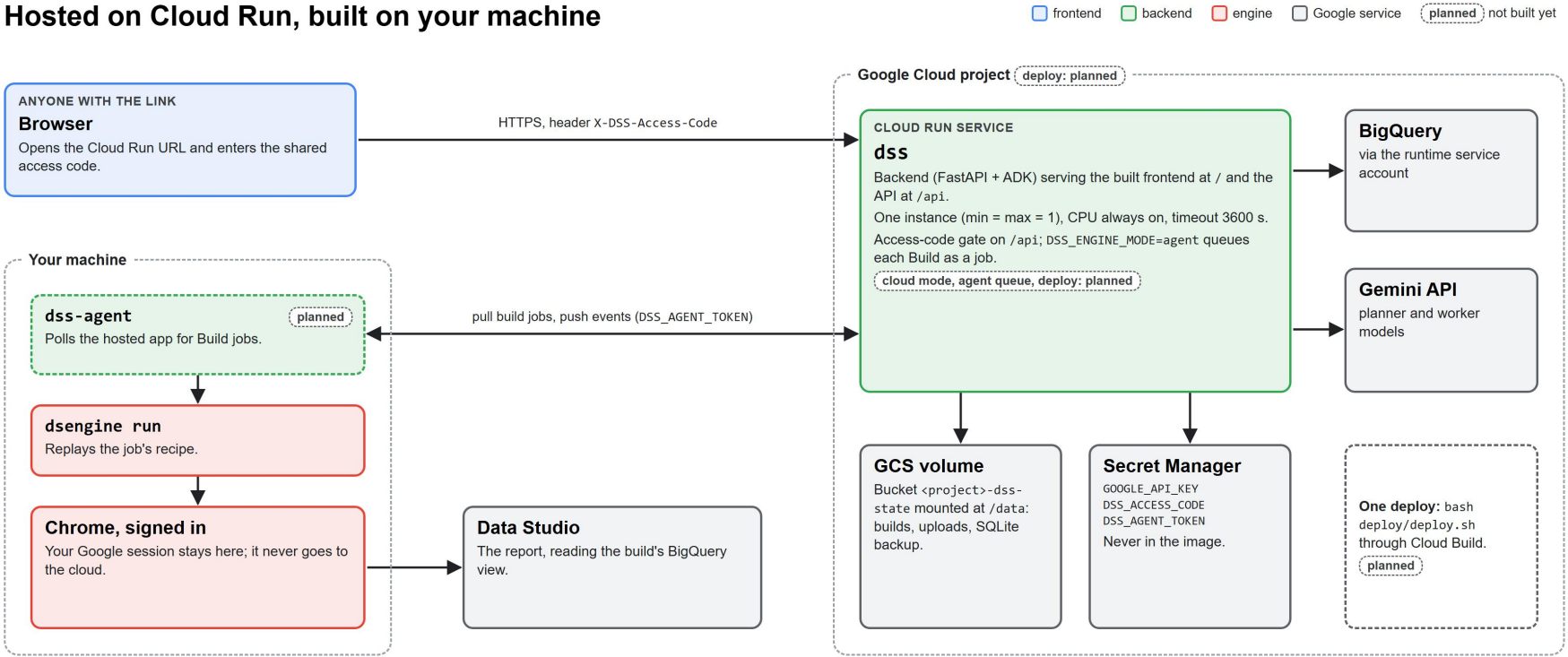
Recipe and engine

The build: a compiled recipe, replayed in Data Studio



Cloud Run deployment

Hosted on Cloud Run, built on your machine



Technologies to be used in the solution

- Google: Gemini API (planner and worker models), BigQuery, Data Studio, Cloud Run, Cloud Storage, Secret Manager, Cloud Build.
- Agent: Google ADK 2.0 Workflow graph; skills as Markdown files; OKF knowledge bundle; SQLite with FTS5.
- Backend: Python 3.12, FastAPI, REST plus server-sent events, uv.
- Engine: Playwright driving a signed-in Chrome.
- Frontend: TypeScript web app built with Vite on Node 20.

Snapshots of the prototype

- TBD: screenshots of the running app (task 4.2): data intake, the interview chat, the commentable HTML prototype, the live build view and the finished Data Studio dashboard.
- Each snapshot is taken from the sample e-commerce dataset (2,000 orders) so the demo is repeatable.

Prototype Performance report/Benchmarking

- TBD: numbers from the end-to-end runs (tasks 2.2, 2.3 and 3.2), each with its run date.
- Planned measures: time from data to dashboard link, model tokens per build, engine steps run and failed, and repair-loop retries.
- All figures will come from the benchmark files in deck/data, never estimates.

Additional Details/Future Development (if any)

- Full ETL for dashboard automation, beyond dashboard creation.
- Community visualizations built by an HTML coding agent (animations, moving map points).
- Audit and rebuild existing Data Studio dashboards with better questions and UX.
- More data sources: Cloud Bigtable and other connectors.
- Deeper LLMOps, security and evals.
- Self-healing patches when Data Studio changes or a user reports a bug.

Provide links to your:

- GitHub public repository: https://github.com/Gjandane/democratizing-google-data-studio
- Demo video (3 minutes): https://drive.google.com/file/d/1eM3ZNkRyDO3jgebuO-JR65NQ-jKZhDsj/view?usp=sharing
- Final product (Cloud Run, access code on request): https://dss-frontend-872000338860.asia-southeast1.run.app

Google Cloud H2S

AI Builder Cup

Build in JAPAC.
Build for the world.

Thank you

